HOTAL MANAGEMENT SYSTEM USING EMU (8086)

SUBMITTED BY:

- Aqsa Sarfraz
- Warda Nadeem
- Ayesha Imtiaz
- Mehak Fatima

SUBMITTED TO:

- Dr. Fatima Anjum

Subject	**:**	Assembly 8086
Semester	**:**	2nd (2026-2029)
Department	**:**	Artificial Intelligence
Submission Date	**:**	1-07-2026

LAHORE COLLEGE FOR WOMEN UNIVERSITY

Context

1. Abstract

The goal of this project is to develop a low-level **Hotel Management System** using 8086 Assembly Language. Moving away from high-level environments, this system directly manipulates registers, stack operations, and system interrupts (INT 21H) to deliver a highly optimized, responsive command-line terminal interface. The program manages room rentals, multiple food categories, handles interactive quantity selections, calculates automated sales reports, and tracks historical bills.

2. INTRODUCTION

In Computer Science and Engineering, understanding low-level computing architectures provides deep insights into performance optimization and resource management. This project serves as a practical implementation of standard x86 Assembly concepts. Instead of relying on bulkier high-level frameworks, this application manages critical business logic—such as user inputs, numeric conversions, data segments, dynamic arithmetic calculations, and system interrupts—directly on the CPU level.

3. PROJECT CONTEXT & PROBLEM STATEMENT

3.1 The Problem

In traditional hospitality environments, managing room orders and food sales using manual paper billing or complex modern software overhead introduces several tracking issues:

- **Resource Inefficiency:** Heavy high-level software requires significant system memory and runtime overhead just to process simple transactions.
- **Data Processing Lag:** Without low-level optimization, calculating multi-item bills, tracking historical quantities, and generating instant sales summaries on older hardware can cause delays.
- **Lack of Direct Hardware Control:** Standard systems rely heavily on third-party frameworks, making them less reliable for dedicated, bare-metal embedded terminal displays.

3.2 Our Solution (The emu 8086 Approach)

To create an ultra-lightweight, high-speed, and secure backend system, we developed a text-based console interface using 8086 Assembly Language. This system interacts directly with processor registers and system interrupts, bypassing heavy graphical layers:

- **Interactive Dynamic Menu:** A clean command-line layout presenting categorized selections (Rooms, Pasta, Burgers, Noodles, Shakes, and Chicken Rolls).

- **Low-Level Validation:** Integrated error traps and conditional jump vectors (CMP, JE, JMP) to catch and isolate invalid character inputs immediately.
- **Integrated Analytics & Automated Billing:** A dedicated live "Sales Information" routine that performs fast 16-bit binary multiplications (MUL) and real-time register additions (ADD) to print precise sales tracking and monetary calculations on demand.

4. SYSTEM ARCHITECTURE & IMPLEMENTATION

4.1 Memory Model & Segmentation

The application utilizes the .MODEL SMALL directive, reserving memory explicitly by defining a single Data Segment (.DATA) and a single Code Segment (.CODE). A dedicated stack space of 256 bytes (.STACK 100H) is allocated to manage hardware interrupts and safely preserve register values during nested execution loops.

4.2 Data Storage & String Structuring

User interfaces and prompt items are hardcoded inside the data segment using Define Byte (DB) modifiers. Carriage returns (13) and line feeds (10) format the layout cleanly in the terminal window, with string structures terminated by the standard x86 string delimiter ($).

4.3 Arithmetic Matrix & Dynamic Pricing

Item costs are directly processed inside 16-bit registers using binary multiplications. The flat pricing structure includes:

- Rooms: 500 units
- Pasta: 300 units
- Burger: 250 units
- Noodles: 200 units
- Shake: 150 units
- Chicken Roll: 180 units
- 4.4 Input Parsing & Digit Conversion

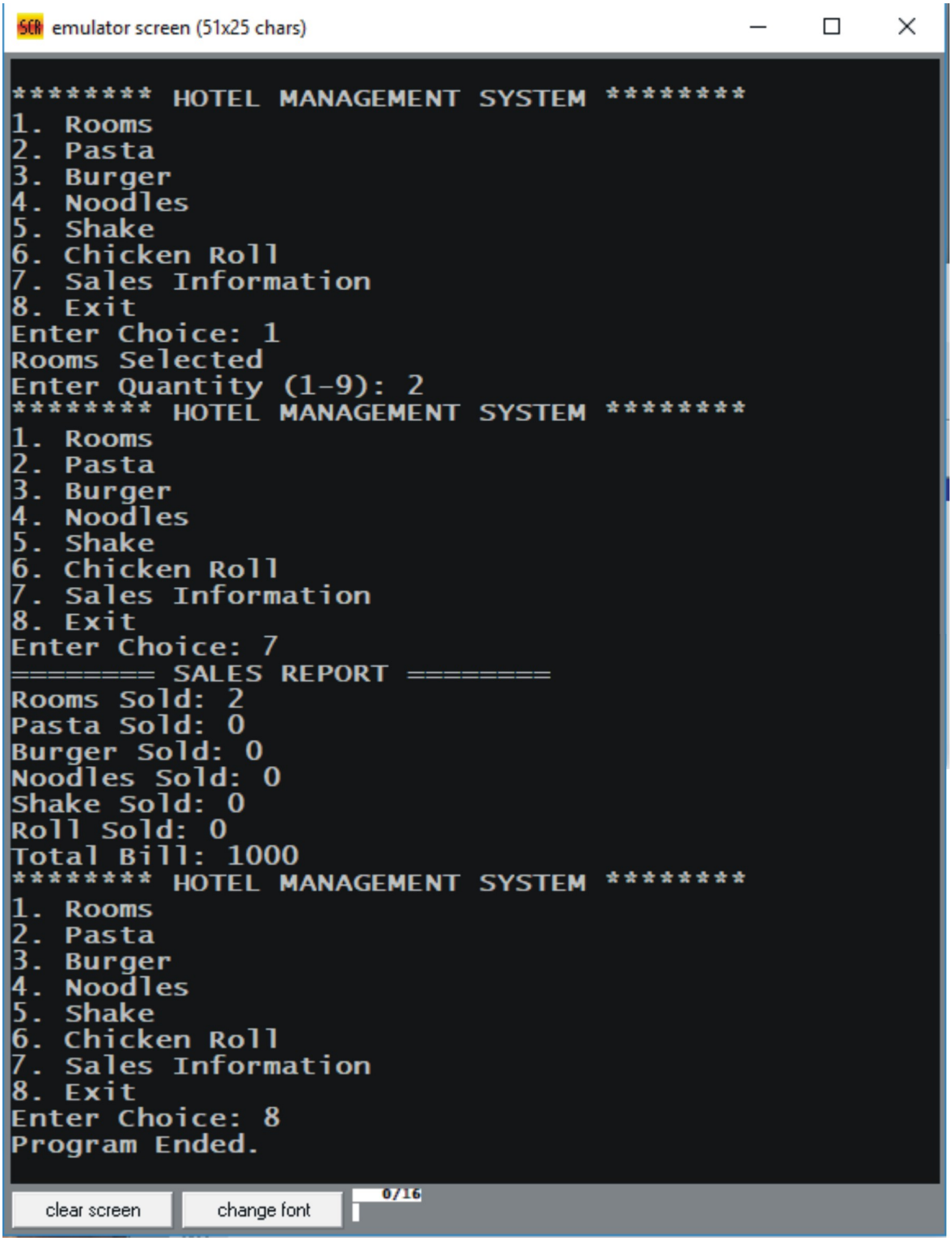
Because character inputs captured via keyboard registers read as raw ASCII values, the system normalizes these inputs by executing an immediate subtraction block (SUB AL, '0'). This strips away the ASCII offset, leaving a pure Integer value that can be cleanly computed by the system multiplier (MUL).

5. TECHNICAL TOOLS & FUNCTIONS

Instruction / Register / Routine	Technical Operation Used	Practical Description inside System
.MODEL SMALL	Memory Segment Directive	Optimizes code execution by setting up exactly one Data Segment and one Code Segment in memory.
INT 21H with AH = 09H	MS-DOS Software Interrupt	Outputs entire text menus, user prompts, and sales labels loaded from the Data Segment until it encounters the $ string terminator.
INT 21H with AH = 01H	Keyboard Input Routine	Halts system execution to wait for the user to press a key, capturing the selection code directly into the AL register.
INT 21H with AH = 02H	Character Display Routine	Outputs an individual processed numeric digit or standard character byte stored inside the DL register onto the console screen.
SUB AL, '0'	Data Normalization	Strips away the ASCII offset hex value from the user's keyboard input, instantly converting a text character into its true integer value for mathematical operations.
CBW	Convert Byte to Word	Safely extends the 8-bit dynamic integer quantity inside the AL register to a full 16-bit word in AX, preventing data truncation and overflows before multiplication.
MUL	Unsigned Binary Multiplication	Multiplies the normalized unit quantities by the pre-configured item base

		prices (e.g., 500 for rooms, 300 for pasta) using the accumulator register.
ADD	Register Accumulation	Accumulates structural inventory data by dynamically adding sales counts and keeping a running real-time tally inside the totalBill memory space.
CMP & JE / JMP	Control Flow Conditional Vectors	Compares raw inputs against execution values to route logic dynamically (e.g., jumping to the PASTA or SALES routines) or catching invalid entries.
DIV in a Loop Block	Numeric-to-ASCII Base-10 Stack Routine	Successively divides high 16-bit totals by 10, pushing individual remainder digits onto the hardware Stack (PUSH) and popping them (POP) to print clean, human-readable numbers on screen.
INT 21H with AH = 4CH	Program Termination Routine	Flushes execution state, releases back hardware registers, and safely transfers program control back over to the operating system shell.

1. TERMINAL INTERFACE LOOK



HOTEL MANAGEMENT SYSTEM

7. CONCLUSION

The **8086 Assembly Hotel Management System** demonstrates how bare-metal instructions can manage structural business logic. By dropping the abstract overhead of modern high-level frameworks and manipulating processors directly through register routing and software interrupts, we created a lightning-fast application. This developmental process allowed our group to master low-level stack tracking, condition branch mappings, numerical register isolation, and real-time data memory segment handling.

8. APPENDIX A: Source Code

This section contains the complete x86 Assembly Language implementation compiled on the Emu8086 emulator for the Hotel Management System.

```
.MODEL SMALL
.STACK 100H

.DATA

msg1 DB 13,10,'******** HOTEL MANAGEMENT SYSTEM ********$'

menu DB 13,10,'1. Rooms'
    DB 13,10,'2. Pasta'
    DB 13,10,'3. Burger'
    DB 13,10,'4. Noodles'
    DB 13,10,'5. Shake'
    DB 13,10,'6. Chicken Roll'
    DB 13,10,'7. Sales Information'
    DB 13,10,'8. Exit'
    DB 13,10,'Enter Choice: $'

roommsg DB 13,10,'Rooms Selected$'
pastamsg DB 13,10,'Pasta Selected$'
burgermsg DB 13,10,'Burger Selected$'
noodlemsg DB 13,10,'Noodles Selected$'
shakemsg DB 13,10,'Shake Selected$'
rollmsg DB 13,10,'Chicken Roll Selected$'

askqty DB 13,10,'Enter Quantity (1-9): $'

invalid DB 13,10,'Invalid Choice!$'
exitmsg DB 13,10,'Program Ended.$'

salesmsg DB 13,10,'======== SALES REPORT ========$'

roomtxt DB 13,10,'Rooms Sold: $'
pastatxt DB 13,10,'Pasta Sold: $'
burgertxt DB 13,10,'Burger Sold: $'
noodletxt DB 13,10,'Noodles Sold: $'
shaketxt DB 13,10,'Shake Sold: $'
rolltxt DB 13,10,'Roll Sold: $'
```

```
billtxt DB 13,10,'Total Bill: $'

qty DB ?

roomCount DW 0
pastaCount DW 0
burgerCount DW 0
noodleCount DW 0
shakeCount DW 0
rollCount DW 0

totalBill DW 0

.CODE

MAIN PROC

MOV AX,@DATA
MOV DS,AX

START:

; print title
LEA DX,msg1
MOV AH,09H
INT 21H

; print menu
LEA DX,menu
MOV AH,09H
INT 21H

; take choice
MOV AH,01H
INT 21H

CMP AL,'1'
JE ROOM
CMP AL,'2'
JE PASTA
CMP AL,'3'
JE BURGER
CMP AL,'4'
JE NOODLE
CMP AL,'5'
JE SHAKE
CMP AL,'6'
JE ROLL
CMP AL,'7'
```

```
JE SALES
CMP AL,'8'
JE EXIT_PROGRAM

; invalid
LEA DX,invalid
MOV AH,09H
INT 21H
JMP START

;================ ROOM ================
ROOM:

LEA DX,roommsg
MOV AH,09H
INT 21H

; ---- GET QTY (inline) ----
LEA DX,askqty
MOV AH,09H
INT 21H

MOV AH,01H
INT 21H
SUB AL,'0'
MOV qty,AL

MOV AL,qty
CBW
ADD roomCount,AX

MOV BX,500
MUL BX
ADD totalBill,AX

JMP START

;================ PASTA ================
PASTA:

LEA DX,pastamsg
MOV AH,09H
INT 21H

LEA DX,askqty
MOV AH,09H
INT 21H

MOV AH,01H
```

```
INT 21H
SUB AL,'0'
MOV qty,AL

MOV AL,qty
CBW
ADD pastaCount,AX

MOV BX,300
MUL BX
ADD totalBill,AX

JMP START

;================ BURGER ================
BURGER:

LEA DX,burgermsg
MOV AH,09H
INT 21H

LEA DX,askqty
MOV AH,09H
INT 21H

MOV AH,01H
INT 21H
SUB AL,'0'
MOV qty,AL

MOV AL,qty
CBW
ADD burgerCount,AX

MOV BX,250
MUL BX
ADD totalBill,AX

JMP START

;================ NOODLE ================
NOODLE:

LEA DX,noodlemsg
MOV AH,09H
INT 21H

LEA DX,askqty
MOV AH,09H
```

```
INT 21H

MOV AH,01H
INT 21H
SUB AL,'0'
MOV qty,AL

MOV AL,qty
CBW
ADD noodleCount,AX

MOV BX,200
MUL BX
ADD totalBill,AX

JMP START

;================ SHAKE ================
SHAKE:

LEA DX,shakemsg
MOV AH,09H
INT 21H

LEA DX,askqty
MOV AH,09H
INT 21H

MOV AH,01H
INT 21H
SUB AL,'0'
MOV qty,AL

MOV AL,qty
CBW
ADD shakeCount,AX

MOV BX,150
MUL BX
ADD totalBill,AX

JMP START

;================ ROLL ================
ROLL:

LEA DX,rollmsg
MOV AH,09H
INT 21H
```

```
LEA DX,askqty
MOV AH,09H
INT 21H

MOV AH,01H
INT 21H
SUB AL,'0'
MOV qty,AL

MOV AL,qty
CBW
ADD rollCount,AX

MOV BX,180
MUL BX
ADD totalBill,AX

JMP START

;================= SALES ==================
SALES:

LEA DX,salesmsg
MOV AH,09H
INT 21H

; ROOM COUNT
LEA DX,roomtxt
MOV AH,09H
INT 21H
MOV AX,roomCount

CALL_PRINT_ROOM:
PUSH AX
PUSH BX
PUSH CX
PUSH DX

MOV BX,10
XOR CX,CX

L1:
XOR DX,DX
DIV BX
PUSH DX
INC CX
CMP AX,0
JNE L1
```

```
L2:
POP DX
ADD DL,'0'
MOV AH,02H
INT 21H
LOOP L2

POP DX
POP CX
POP BX
POP AX

; PASTA
LEA DX,pastatxt
MOV AH,09H
INT 21H
MOV AX,pastaCount

PUSH AX
PUSH BX
PUSH CX
PUSH DX

MOV BX,10
XOR CX,CX

L3:
XOR DX,DX
DIV BX
PUSH DX
INC CX
CMP AX,0
JNE L3

L4:
POP DX
ADD DL,'0'
MOV AH,02H
INT 21H
LOOP L4

POP DX
POP CX
POP BX
POP AX

; BURGER
LEA DX,burgertxt
```

```
MOV AH,09H
INT 21H
MOV AX,burgerCount

PUSH AX
PUSH BX
PUSH CX
PUSH DX

MOV BX,10
XOR CX,CX

L5:
XOR DX,DX
DIV BX
PUSH DX
INC CX
CMP AX,0
JNE L5

L6:
POP DX
ADD DL,'0'
MOV AH,02H
INT 21H
LOOP L6

POP DX
POP CX
POP BX
POP AX

; NOODLE
LEA DX,noodletxt
MOV AH,09H
INT 21H
MOV AX,noodleCount

PUSH AX
PUSH BX
PUSH CX
PUSH DX

MOV BX,10
XOR CX,CX

L7:
XOR DX,DX
DIV BX
```

```
PUSH DX
INC CX
CMP AX,0
JNE L7

L8:
POP DX
ADD DL,'0'
MOV AH,02H
INT 21H
LOOP L8

POP DX
POP CX
POP BX
POP AX

; SHAKE
LEA DX,shaketxt
MOV AH,09H
INT 21H
MOV AX,shakeCount

PUSH AX
PUSH BX
PUSH CX
PUSH DX

MOV BX,10
XOR CX,CX

L9:
XOR DX,DX
DIV BX
PUSH DX
INC CX
CMP AX,0
JNE L9

L10:
POP DX
ADD DL,'0'
MOV AH,02H
INT 21H
LOOP L10

POP DX
POP CX
POP BX
```

```
POP AX

; ROLL
LEA DX,rolltxt
MOV AH,09H
INT 21H
MOV AX,rollCount

PUSH AX
PUSH BX
PUSH CX
PUSH DX

MOV BX,10
XOR CX,CX

L11:
XOR DX,DX
DIV BX
PUSH DX
INC CX
CMP AX,0
JNE L11

L12:
POP DX
ADD DL,'0'
MOV AH,02H
INT 21H
LOOP L12

POP DX
POP CX
POP BX
POP AX

; TOTAL BILL
LEA DX,billtxt
MOV AH,09H
INT 21H
MOV AX,totalBill

PUSH AX
PUSH BX
PUSH CX
PUSH DX

MOV BX,10
XOR CX,CX
```

```
L13:
XOR DX,DX
DIV BX
PUSH DX
INC CX
CMP AX,0
JNE L13

L14:
POP DX
ADD DL,'0'
MOV AH,02H
INT 21H
LOOP L14

POP DX
POP CX
POP BX
POP AX

JMP START

;================ EXIT ================
EXIT_PROGRAM:

LEA DX,exitmsg
MOV AH,09H
INT 21H

MOV AH,4CH
INT 21H

MAIN ENDP
END MAIN
```